

Package ‘micsPlusTableR’

September 19, 2026

Title MICS Plus Excel Tabulation and Review Application

Version 0.3.1

Description Runs interactive and scriptable workflows for preparing, tabulating, checking, reviewing, and exporting MICS Plus tables from an Excel tabulation plan, survey data files, and a survey-specific preparation script.

License MIT + file LICENSE

Encoding UTF-8

Roxygen list(markdown = TRUE)

RoxygenNote 7.3.3

Depends R (>= 4.2.0)

Imports bslib, dplyr, flextable, haven, here, htmltools, htmlwidgets, janitor, jsonlite, later, officer, openxlsx2, purrr, reactable, readr, readxl, rlang, shiny, shinyFiles, stringr, tibble, tidyr, tidyxl, ggplot2, Hmisc, labelled, memisc, reshape2, RM.weights, survey, srvyr

Suggests testthat (>= 3.0.0), knitr, rmarkdown

Config/micsPlusTableR/prep-packages ggplot2, Hmisc, labelled, memisc, reshape2, RM.weights, survey, srvyr

VignetteBuilder knitr

Config/testthat/edition 3

NeedsCompilation no

Config/build/clean-inst-doc FALSE

Contents

micsPlusTableR-package	2
as_sdmx_compatible	5
calc_cells	5
check_mics_table	6
check_prep_dependencies	7
col_condition_f	7
col_group_perc_total_check	8
col_group_total_check	9
compare_mics_tables	9
create_excel_workbooks	10

download_survey_files	11
extract_var	12
filter_second	13
get_blank_cols	13
install_prep_dependencies	14
list_survey_files	15
mics_session	16
mics_table_name	16
normalize_condition_text	17
offline-workflow	17
package-dependencies	18
pivot_mics_table	19
prep-script-dependencies	19
prepare_mics_data	20
read_mics_tabulation	21
read_previous_mics_table	22
row_condition_f	22
row_group_perc_total_check	23
row_group_total_check	23
row_indent_group_perc_total_check	24
row_indent_group_total_check	25
run_app	25
statistic-precision	26
survey_choices	27
tabulate_extra_table	28
tabulate_h	28
tabulate_mics_table	29
tabulate_v	30
validate_sdmx_compatible	30
write_mics_footnotes	31
write_mics_table	31
Index	33

micsPlusTableR-package

micsPlusTableR: MICS Plus Excel tabulation and review

Description

Prepare, tabulate, check, review, and export MICS Plus survey tables using an Excel tabulation plan. Work interactively with `run_app()` or use the scripting functions with an isolated `mics_session()`.

Inputs

Supply an Excel tabulation plan with an IDX worksheet and a worksheet for each table, household and household-member SPSS files, and a survey-specific R preparation script that reads `hh_path` and `hl_path` and creates data frames named `hh` and `hl`. A sibling `FIES-inputs` directory is optional.

User setup

Install R and an editor such as RStudio Desktop. Before installing this package or starting the app, create or open your own survey project. In RStudio, use File > New Project for a new or existing folder, or File > Open Project for an existing .Rproj file. Install the package with `remotes::install_github("barisr/micsPlusTableR")` after installing remotes, then restart R. The bundled `doc/user-guide.pdf` and `doc/user-guide.html` explain computer setup, project creation, installation, and file uploads for beginners. The app's User's Guide tab opens the guide after launch.

Positron, VS Code, Cursor, Jupyter with an R kernel, and R's own console can also run the package. Configure R support and start R in the survey folder. Check `getwd()` and `here::here()` or set `output_dir` explicitly; use `launch.browser = TRUE` if needed. Remote sessions need browser routing and access to server files. Users do not need the maintainer's project.

Interactive workflow

Reopen the survey project on each visit; installation is normally needed only once per R library. Select inputs again for each app session. Start `run_app()`. In Download Files, load the list and download one, several, or all survey folders to the current project or a chosen location. This saves plans and preparation materials; household/member data must be obtained from <https://mics.unicef.org/surveys> with permission. In Data Preparation, select the survey and wave, provide the inputs, prepare the data, calculate tables, review checks, and export the results. The app creates `micsPlusTableR-output` in the project root by default. To select another location, pass the `output_dir` argument to `run_app()`.

Scripting workflow

The main steps are:

- `mics_session()` creates an isolated session.
- `prepare_mics_data()` runs the preparation script.
- `tabulate_mics_table()` calculates cell results.
- `check_mics_table()` checks consistency.
- `pivot_mics_table()` arranges results for review.
- `create_excel_workbooks()` creates output workbooks.
- `write_mics_table()` writes the results.
- `write_mics_footnotes()` writes table footnotes.
- `read_previous_mics_table()` reads an earlier output.
- `compare_mics_tables()` compares two outputs.

See [offline-workflow](#) for a complete scripting example.

Individual steps and testing

`tabulate_v()` and `tabulate_h()` calculate a loaded plan in a specific direction; `calc_cells()` tests cells against explicit input data. `tabulate_extra_table()` maps supplied values to the loaded plan. `row_condition_f()` parses row predicates. `col_condition_f()` parses column predicates. `normalize_condition_text()` normalizes Excel text. `extract_var()` identifies variables. `get_blank_cols()` supports worksheet cleanup. `filter_second()` supports comparison cleanup. Run a single consistency check:

- `row_group_total_check()` checks row counts.
- `row_group_perc_total_check()` checks row percentages.
- `col_group_total_check()` checks column counts.
- `col_group_perc_total_check()` checks column percentages.
- `row_indent_group_total_check()` checks indented counts.
- `row_indent_group_perc_total_check()` checks indented percentages.

Each returns status, issue count, all comparisons, and failing comparisons. Stateful helpers take a session; see the getting-started vignette for examples.

Dependencies

[package-dependencies](#) explains required app/engine packages and development tools. [prep-script-dependencies](#) documents survey packages separately. Both dependency groups install with the package; no separate preparation setup is needed.

For diagnosis, use `check_prep_dependencies()`. For repairs or extra custom-script requirements, use `install_prep_dependencies()`.

Survey and wave configuration

Maintain `inst/extdata/survey_choices.csv` in the package source. It remains a plain, editable CSV on GitHub and is bundled automatically on installation. See [survey_choices](#) for its columns, validation rules, and update workflow.

Outputs and interpretation

Outputs include Excel workbooks and long-format observations. The app offers Excel, CSV, and RDS downloads of the full prepared long-format dataset. Use `as_sdmx_compatible()` to convert observations to the project's SDMX-compatible contract. Use `validate_sdmx_compatible()` to check them. This contract does not by itself establish compliance with an external SDMX data structure. Table checks assess consistency; analysts remain responsible for preparation scripts, survey design, denominators, and interpretation of estimates.

Author(s)

Maintainer: MICS Plus Tabulation Team <micsplustabler@example.org>

See Also

[run_app\(\)](#), [survey_choices](#), [offline-workflow](#)

Examples

```
session <- mics_session()
session
system.file("extdata", "survey_choices.csv", package = "micsPlusTableR")
```

as_sdmx_compatible	<i>Add an SDMX-compatible observation profile to MICS cell results</i>
--------------------	--

Description

Converts either the package's engine cell results or an already standardized MICS cell table into a 35-variable long-format contract adapted from the mics-translation project. The output omits the custom table_id, value, estimate, digits, and unw duplicates; use TABLE_ID, OBS_VALUE, DECIMALS, and denominator_n instead. Input fields are unchanged. Source coordinates remain in row_order and column_order; the redundant row_id, column_id, ROW_ID, and COLUMN_ID fields are omitted. The output also omits unweighted_n, which duplicates denominator_n for engine cells. For standardized input, the supplied denominator_n is retained. The optional input unit_measure override is applied to UNIT_MEASURE but is not included as a separate output field.

Usage

```
as_sdmx_compatible(cells, config, table_id = NULL, table_context = NULL)
```

Arguments

cells	A data frame with one row per table cell.
config	A list containing frequency, ref_area, survey_id, and time_period.
table_id	Table identifier. Required for raw engine cell results.
table_context	Optional parsed tabulation context used to recover multi-level column headers and suppression settings.

Value

A tibble with one row per observation and the 35 standardized fields.

calc_cells	<i>Calculate individual horizontal table cells</i>
------------	--

Description

Calculate individual horizontal table cells

Usage

```
calc_cells(df, tab_r, tab_c3, tab, weight_var, weighted = FALSE)
```

Arguments

df	Prepared data frame, after block filters and calculations.
tab_r	Row specification with row_index and row_lgc.
tab_c3	Column specification with col_index, col_condition, and col_var_name; optional col_logic preserves display labels.
tab	Cell specification with row_index, col_index, and stat_type.
weight_var	Weight-column name. May be NA for unweighted calculations.
weighted	FALSE for unweighted results, TRUE for weighted results, or "both" (legacy mode, returns the unweighted result only).

Details

Supports `n/n1`, `n_unw/n_unw1`, `p/p1/mean`, `mean(variable)`, `median(variable)`, `p_sum(variable)`, and `100`. Median and Mean -prefixed indicators are unweighted. `p_sum` always uses the supplied weight column. Missing statistic cells return NA. An empty denominator can produce NaN. Optional `d=` arguments are returned as `display_digits` metadata and never round the calculated value; see [statistic-precision](#).

Value

A tibble with row/column indices, logic, statistic, and numeric value.

Examples

```
calc_cells(data.frame(total = c(1, 1)),
  data.frame(row_index = 9L, row_lgc = "TRUE"),
  data.frame(col_index = 3L, col_condition = "total == 1", col_var_name = "total_1"),
  data.frame(row_index = 9L, col_index = 3L, stat_type = "n_unw"),
  weight_var = NA_character_)
```

check_mics_table	<i>Run all applicable consistency checks</i>
------------------	--

Description

Run all applicable consistency checks

Usage

```
check_mics_table(session, table = NULL, tolerance = 1e-06)
```

Arguments

session	A tabulation session.
table	Cell results; defaults to the session's latest result.
tolerance	Maximum absolute difference treated as consistent.

Value

A list containing a summary data frame and detailed check tables.

 check_prep_dependencies

Check dependencies of a preparation script and its helpers

Description

Inspect preparation code without executing it, then check whether each required package can be loaded. App dependencies and survey-specific preparation dependencies install together and are documented separately in [prep-script-dependencies](#). This is a diagnostic tool; preparation checks dependencies automatically.

Usage

```
check_prep_dependencies(prepare_script, stop_on_missing = FALSE)
```

Arguments

`prepare_script` Path to a preparation R script.

`stop_on_missing` Stop with an installation command if a required package is missing or cannot be loaded. Defaults to FALSE for inspection.

Details

Reads the script's literal `prep_dependencies` declaration, qualified calls, `library/require` calls with literal package names, and known legacy survey/FIES function calls. It also inspects all R files in the sibling FIES-inputs folder and recursively follows literal source paths. Dynamically computed package names and source paths require an explicit `prep_dependencies` declaration in the main script. This checks package availability, not numerical correctness or version compatibility of the survey code. It never executes preparation code.

Value

A data frame with `package`, `required_by` (script basenames), `available`, and `problem` (NA when loadable).

See Also

[install_prep_dependencies\(\)](#), [prepare_mics_data\(\)](#)

 col_condition_f

Parse column conditions from a plan

Description

Parse column conditions from a plan

Usage

```
col_condition_f(tab_c)
```

Arguments

tab_c Data frame containing col_index and col_lgc.

Details

Uses the last meaningful line after excluding source, filter, calculation, separator, and weight lines. Parses text without evaluating it against survey data.

Value

A tibble containing the column index, original condition, cleaned condition, and generated variable name.

See Also

[row_condition_f\(\)](#), [normalize_condition_text\(\)](#)

col_group_perc_total_check

Check that column-block percentages sum to 100

Description

Check that column-block percentages sum to 100

Usage

```
col_group_perc_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use [check_mics_table\(\)](#) to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

col_group_total_check *Compare column-group counts with the overall total*

Description

Compare column-group counts with the overall total

Usage

```
col_group_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use [check_mics_table\(\)](#) to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

compare_mics_tables *Compare two generated tables*

Description

Compare two generated tables

Usage

```
compare_mics_tables(
  previous,
  current,
  ignore_col_names = FALSE,
  show_diff_over = 0
)
```

Arguments

previous	Previous or approved table.
current	Current table.
ignore_col_names	Compare columns by position instead of name.
show_diff_over	Minimum absolute numeric difference to report.

Value

A data frame containing differing cells.

create_excel_workbooks

Create output workbooks from a tabulation plan

Description

Copies the tabulation plan to standard output names without overwriting an existing file. A numeric suffix is added after a filename collision.

Usage

```
create_excel_workbooks(
  path_tab_excel,
  output_dir,
  country_code,
  period,
  wave,
  target = c("both", "output", "formatted"),
  time_tag = format(Sys.time(), "%Y%m%d%H%M%S")
)
```

Arguments

path_tab_excel	Existing .xls or .xlsx tabulation plan.
output_dir	Directory where workbooks will be created.
country_code	Short country code used in filenames.
period	Survey period used in filenames.
wave	Wave label used in filenames.
target	One of "output", "formatted", or "both".
time_tag	Timestamp used in filenames.

Value

A list with output and formatted paths.

download_survey_files *Download survey tabulation plans and preparation files*

Description

Save selected plans, preparation scripts, and supporting reference assets from the public survey files repository. Household and household-member microdata must be obtained separately with the required permission.

Usage

```
download_survey_files(
  surveys = NULL,
  files = NULL,
  dest_dir = "inputs",
  overwrite = FALSE,
  quiet = FALSE,
  repo = "barisr/micsPlusTableR-Files",
  ref = "main"
)
```

Arguments

surveys	Character vector of root survey folder names, such as "JAM_W1". NULL selects all surveys.
files	Character vector of exact repository-relative paths from <code>list_survey_files()</code> . NULL selects all files in the selected surveys.
dest_dir	Destination folder, relative to the current R working directory unless an absolute path is supplied. Defaults to "inputs".
overwrite	Replace existing files? Defaults to FALSE; any existing selected file stops the operation before downloading file contents.
quiet	Suppress file download progress and the completion message?
repo	GitHub repository in owner/name form.
ref	Branch, tag, or commit to download. Defaults to "main".

Details

Survey folders use a three-letter country code and wave, such as JAM_W1. Downloads include root-level Excel plans (.xls, .xlsx) and .R scripts or .RData reference assets inside prep-files-* subfolders. Other files, including SPSS microdata and hidden files, are excluded. Preparation subfolders are preserved. A single Git commit is resolved before listing or downloading, so changes to the branch during a download cannot mix remote versions. Newly published survey folders are discovered without a package update.

Downloads are staged and their sizes checked before destination files are written. A network failure leaves existing destination files unchanged. Files are downloaded in binary mode and are never executed. Selecting individual files may omit preparation helpers; select a whole survey folder for all its published materials. Microdata must still be obtained separately. Git LFS pointers are not supported.

Internet access is required. Unauthenticated GitHub API rate limits apply; if a request fails, check the connection and public repository/ref, or retry later. The browser ZIP link in the user's guide is an alternative.

Value

Invisibly returns the selected file-list rows with survey, path, size, url, and local_path columns, plus the resolved commit attribute.

See Also

[list_survey_files\(\)](#)

Examples

```
## Not run:
download_survey_files("JAM_W1")
download_survey_files(c("JAM_W1", "MNG_W2"), dest_dir = "survey-inputs")
download_survey_files() # All surveys
available <- list_survey_files()
download_survey_files(files = available$path[1])

## End(Not run)
```

extract_var

Extract the leading variable from a condition

Description

Extract the leading variable from a condition

Usage

```
extract_var(x)
```

Arguments

x One condition string.

Value

A character scalar naming the left-hand operand of a comparison, between, or membership expression; NA for unsupported or invalid syntax.

filter_second	<i>Remove rows with a missing second-column value</i>
---------------	---

Description

Remove rows with a missing second-column value

Usage

```
filter_second(.)
```

Arguments

. A data frame with at least two columns.

Value

The input data frame with rows containing NA in column two removed.

Examples

```
filter_second(data.frame(label = c("A", "B"), value = c(NA, 3)))
```

get_blank_cols	<i>Find blank worksheet header columns</i>
----------------	--

Description

Find blank worksheet header columns

Usage

```
get_blank_cols(tab)
```

Arguments

tab Wide data frame containing row_index and Excel columns named 1, 2, etc. Header rows 3 through 8 are inspected.

Value

Sorted integer column indices, excluding column 1. NA, empty text, and the string NA count as blank.

`install_prep_dependencies`*Install missing preparation-script dependencies*

Description

Install missing preparation-script dependencies

Usage

```
install_prep_dependencies(  
  prep_script,  
  repos = "https://cloud.r-project.org",  
  lib = .libPaths()[[1L]]  
)
```

Arguments

<code>prep_script</code>	Path to a preparation R script.
<code>repos</code>	Package repository URL, defaulting to the CRAN cloud mirror.
<code>lib</code>	Installation library. Defaults to the first active library. It must be listed in <code>.libPaths()</code> .

Details

Maintained survey dependencies install with `micsPlusTableR`. This function is only needed for repair or extra custom-script packages.

[check_prep_dependencies\(\)](#) identifies missing or unloadable packages. Only those packages are installed. Transitive required dependencies are installed by R. Preparation scripts are never executed by this function.

Installation happens only when this function is explicitly called.

[prepare_mics_data\(\)](#) reports missing packages without installing them.

Value

The dependency check data frame, invisibly, after installation.

See Also

[check_prep_dependencies\(\)](#), [prep-script-dependencies](#)

list_survey_files	<i>List survey tabulation plans and preparation files</i>
-------------------	---

Description

Discover or download tabulation plans, preparation scripts, and their supporting reference assets from the separate survey files repository. Household and household-member microdata are not distributed here; obtain them separately from the MICS Plus website with the required permission. The repository must be public. No GitHub account or token is needed for these plans and preparation files.

Usage

```
list_survey_files(repo = "barisr/micsPlusTableR-Files", ref = "main")
```

Arguments

repo	GitHub repository in owner/name form.
ref	Branch, tag, or commit to download. Defaults to "main".

Details

Survey folders use a three-letter country code and wave, such as JAM_W1. Downloads include root-level Excel plans (.xls, .xlsx) and .R scripts or .RData reference assets inside prep-files-* subfolders. Other files, including SPSS microdata and hidden files, are excluded. Preparation subfolders are preserved. A single Git commit is resolved before listing or downloading, so changes to the branch during a download cannot mix remote versions. Newly published survey folders are discovered without a package update.

Downloads are staged and their sizes checked before destination files are written. A network failure leaves existing destination files unchanged. Files are downloaded in binary mode and are never executed. Selecting individual files may omit preparation helpers; select a whole survey folder for all its published materials. Microdata must still be obtained separately. Git LFS pointers are not supported.

Internet access is required. Unauthenticated GitHub API rate limits apply; if a request fails, check the connection and public repository/ref, or retry later. The browser ZIP link in the user's guide is an alternative.

Value

list_survey_files() returns a data frame with survey, path, size (bytes), and url columns, plus a commit attribute.

See Also

[download_survey_files\(\)](#)

Examples

```
## Not run:
available <- list_survey_files()
unique(available$survey)
available$path

## End(Not run)
```

mics_session	<i>Create a non-Shiny MICS Plus tabulation session</i>
--------------	--

Description

Create a non-Shiny MICS Plus tabulation session

Usage

```
mics_session(hh = NULL, hl = NULL)
```

Arguments

hh	Optional prepared household data frame.
hl	Optional prepared household-member data frame.

Value

A mutable, isolated tabulation session environment.

mics_table_name	<i>Get the current table title</i>
-----------------	------------------------------------

Description

Get the current table title

Usage

```
mics_table_name(session)
```

Arguments

session	A session with a tabulation sheet already read.
---------	---

Value

The table title as a character scalar.

normalize_condition_text

Normalize condition text copied from Excel

Description

Normalize condition text copied from Excel

Usage

```
normalize_condition_text(x)
```

Arguments

x Character vector of conditions.

Value

Trimmed text with smart quotes, nonbreaking spaces, and Unicode comparison/logical operators replaced by their R equivalents; NA is retained.

offline-workflow

Run MICS Plus tabulation workflows outside Shiny

Description

These functions provide the supported non-Shiny interface to the packaged tabulation engine. The explicit session keeps survey data, plan context, extra tables, and calculated results together without writing them into the user's global workspace.

Details

Prepare survey data in a new session, calculate a table from the Excel plan, check its consistency, and pivot it for review before writing results. The example requires analyst-supplied files. See the package overview for links to the function references and workbook export steps.

See Also

[micsPlusTableR-package](#)

Examples

```
## Not run:
session <- mics_session()
prepare_mics_data(
  session,
  "hh.sav",
  "hl.sav",
  prep_script = "survey-preparation/survey_prep.R",
  output_dir = "micsPlusTableR-output"
)
```

```

result <- tabulate_mics_table(session, "TabulationPlan.xlsx", "1.1")
checks <- check_mics_table(session, result)
preview <- pivot_mics_table(session, result, type = "header")

## End(Not run)

```

package-dependencies *Package runtime and development dependencies*

Description

Required Imports cover the app, tabulation engine, and maintained preparation scripts. They install together. Preparation-only packages are documented separately in [prep-script-dependencies](#).

Details

Runtime packages and their purposes:

- shiny, bslib, shinyFiles: application UI, server, and file selection.
- htmltools, htmlwidgets, reactable: HTML controls and table displays.
- flextable, officer: formatted table previews and borders.
- openxlsx2: Excel output workbooks and formatting.
- readxl, tidyxl: worksheet values, cell formats, and plan parsing.
- dplyr, tidyr, tibble: table transformation and cell results.
- purrr, rlang, stringr: iteration, expressions, and condition parsing.
- haven: display and clean labelled survey values in the app.
- janitor: remove empty columns from comparison workbooks.
- here: locate the user's project and default output folder.
- jsonlite: read the remote survey-file listing.
- readr: CSV downloads of long-format results.
- later: schedule application UI updates.

Development-only Suggests are testthat for tests and knitr/rmarkdown for vignette building. Developer tools such as roxygen2, pkgload, and remotes are installed separately as described in the README.

The maintainer command `Rscript tools/check-dependencies.R` inspects executable app and engine code, qualified references, and imported function calls. It rejects undeclared direct references and Imports with no detected usage in the app, engine, or preparation namespace loaders. It also checks that all documented preparation dependencies are required Imports. Both package and reference-manual builds run this audit.

See Also

[prep-script-dependencies](#), [micsPlusTableR-package](#)

pivot_mics_table	<i>Pivot long-format cell results for review</i>
------------------	--

Description

Pivot long-format cell results for review

Usage

```

pivot_mics_table(
  session,
  table = NULL,
  formatted = FALSE,
  type = c("index", "header", "logic")
)

```

Arguments

session	A tabulation session.
table	Cell results; defaults to the session's latest result.
formatted	FALSE, TRUE, or "view" as supported by the engine.
type	One of "index", "header", or "logic".

Value

A wide review table.

prep-script-dependencies	<i>Preparation script dependencies</i>
--------------------------	--

Description

Survey preparation packages are tracked separately from the packages needed to launch the app and run the tabulation engine. Both groups are required Imports and install together with mic-sPlusTableR; no separate setup is needed.

Details

The maintained preparation scripts declare `prep_dependencies` near the top of the file.

`prepare_mics_data()` checks these packages and the detected helper dependencies before executing the script. The Shiny Data Preparation workflow uses the same check. Missing or unloadable packages produce an error naming the package, the script requiring it, and an installation command.

Use `check_prep_dependencies()` for diagnosis.

Use `install_prep_dependencies()` for repair or extra custom-script packages. Neither is a required step for normal installation or switching between maintained surveys.

Additional DESCRIPTION field: `Config/micsPlusTableR/prep-packages`.

Preparation-only packages in the maintained Jamaica workflows:

- `ggplot2`: FIES distribution and thermometer plots.
- `Hmisc`: weighted prevalence means in the FIES helpers.
- `memisc`: SPSS import retaining value labels for the FIES routines.
- `reshape2`: reshape FIES plotting data with `melt`.
- `RM.weights`: weighted Rasch model and food-insecurity calculations.
- `survey`: survey designs and design-based FIES estimates.
- `srvyr`: survey-design pipelines in the main preparation scripts.
- `labelled`: variable/value labels in Jamaica Wave 2 recodes.

All eight packages are installed by the normal package installation command. They are not imported into the app's namespace. Packages required by the selected script are made available in its private preparation environment; package functions already supplied by the core workflow retain precedence. Use `explicit package::function` calls if a script needs a conflicting name.

Shared dependencies are also declared by the scripts: Mongolia Waves 1 and 2 use `dplyr` and `haven`. Jamaica Waves 1 and 2 also use `here`, `tibble`, and `tidyr`; Wave 2 additionally uses `purrr`. These are already package Imports.

Keep declarations current when editing scripts or adding helpers. The checker scans the main script, all R files in its sibling FIES-inputs directory, and literal source paths. Declare packages used through dynamically constructed names or source paths explicitly. The checker does not execute preparation code and does not certify statistical results or package-version compatibility.

See Also

[prepare_mics_data\(\)](#), [package-dependencies](#)

Examples

```
## Not run:
prep_script <- "survey-preparation/survey_prep.R"
check_prep_dependencies(prepare_script)
# For repair or additional dependencies in a custom script only:
install_prep_dependencies(prepare_script)

## End(Not run)
```

```
prepare_mics_data
```

```
Run a survey preparation script outside the Shiny application
```

Description

Run a survey preparation script outside the Shiny application

Usage

```
prepare_mics_data(session, hh_path, hl_path, prep_script, output_dir = NULL)
```

Arguments

session	A session created by <code>mics_session()</code> .
hh_path	Path to the household .sav file.
hl_path	Path to the household-member .sav file.
prep_script	Path to the trusted preparation .R script. A sibling FIES-inputs folder is detected automatically.
output_dir	Main output folder. If the script's folder contains FIES-inputs, FIES artifacts are written to FIES-outputs below this folder.

Details

Checks declared and detected preparation dependencies before running any preparation code. Maintained survey dependencies install with the package. `install_prep_dependencies()` is available for repair or extra custom-script requirements. See `prep-script-dependencies` for the separate dependency list.

Value

The session, invisibly.

`read_mics_tabulation` *Read one sheet from a MICS Plus tabulation plan*

Description

Read one sheet from a MICS Plus tabulation plan

Usage

```
read_mics_tabulation(session, path_tab_excel, sheet)
```

Arguments

session	A session created by <code>mics_session()</code> .
path_tab_excel	Path to the Excel tabulation plan.
sheet	Sheet name or index.

Value

The parsed tabulation-plan context.

```
read_previous_mics_table
```

Read a previously generated worksheet for comparison

Description

Read a previously generated worksheet for comparison

Usage

```
read_previous_mics_table(path, sheet, skip = 3)
```

Arguments

path	Excel workbook path.
sheet	Sheet name or index.
skip	Number of header rows to skip.

Value

A cleaned table.

```
row_condition_f
```

Parse row conditions from a plan

Description

Parse row conditions from a plan

Usage

```
row_condition_f(tab_r)
```

Arguments

tab_r	Data frame containing row_index and row_lgc.
-------	--

Details

Separates mutate calls from predicates; blank or NA row logic means TRUE. Parses text without evaluating it against survey data.

Value

A tibble containing the row index, original condition, cleaned condition, generated variable name, and calculation.

See Also

[col_condition_f\(\)](#), [normalize_condition_text\(\)](#)

row_group_perc_total_check

Check that row-group percentages sum to 100

Description

Check that row-group percentages sum to 100

Usage

```
row_group_perc_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use [check_mics_table\(\)](#) to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

row_group_total_check *Compare row-group counts with the overall total*

Description

Compare row-group counts with the overall total

Usage

```
row_group_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use `check_mics_table()` to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

row_indent_group_perc_total_check
<i>Compare indented child percentages with their parent row</i>

Description

Compare indented child percentages with their parent row

Usage

```
row_indent_group_perc_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use `check_mics_table()` to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

row_indent_group_total_check

Compare indented child counts with their parent row

Description

Compare indented child counts with their parent row

Usage

```
row_indent_group_total_check(session, cell_results = NULL, diff = 1e-11)
```

Arguments

session	A tabulation session. Indent checks also require the loaded plan's indent_rows metadata.
cell_results	Long-format numeric results, defaulting to the latest result in the session. Use the complete results with logic/group metadata.
diff	Maximum absolute difference accepted as consistent.

Details

Runs only this check and updates its corresponding session check state. A missing total or distribution produces not_applicable. Use [check_mics_table\(\)](#) to run all six checks together.

Value

A list with status (pass, fail, or not_applicable), issue_count, details (all comparisons), and issues (failing comparisons).

run_app

Run the micsPlusTableR Shiny application

Description

Starts the Shiny application bundled with micsPlusTableR. The Download Files tab saves survey plans and preparation files to the current project by default; its destination can be changed in the tab.

Usage

```
run_app(
  host = "127.0.0.1",
  port = getOption("shiny.port"),
  launch.browser = interactive(),
  output_dir = NULL,
  ...
)
```

Arguments

host	Host interface passed to shiny::runApp().
port	Port passed to shiny::runApp(); NULL selects a free port.
launch.browser	Whether to open the application in a browser.
output_dir	Default directory for generated Excel workbooks. When NULL, the app creates micsPlusTableR-output in the project root detected by here::here().
...	Additional arguments passed to shiny::runApp().

Value

Invisibly returns the value from shiny::runApp().

statistic-precision	<i>Worksheet display precision</i>
---------------------	------------------------------------

Description

Every supported worksheet statistic accepts an optional named d argument specifying the number of decimal places to display.

Details

Use mean(HCS8, d=0) for a whole-number expense mean, mean(d=1) for a mean indicator with one decimal place, p(d=2) for two percentage decimals, or n(d=1) for a count with one displayed decimal. Existing argument forms retain their arguments: median(age, d=2), mean_unw(age, d=0), p_sum(x, d=2), and p(100, d=2). A constant may use 100(d=2). The statistic must be supported in the table's direction; adding d does not change the calculation or its weighting.

d must be a literal non-negative whole number, supplied once. Zero means no decimal point. Omit d to keep the existing formatting for that statistic and output mode; means still default to one decimal place. Spaces around arguments are allowed. d means decimal places, not significant figures.

The Formatted and Suppressed Tabulator views honor explicit precision for visible numbers; Unformatted retains full numeric estimates. Excel applies number formats in both output workbooks, including multi-sheet runs. Parentheses retain the requested precision; (*) and - remain markers. Display rounding does not alter stored estimates, calculations, weights, denominators, suppression thresholds, or consistency checks.

Internally, stat_type contains the statistic without d; display_digits holds the explicit integer or NA when omitted. Worksheet filling carries the statistic and its precision together. Changing only d does not change mean filter boundaries or the identification of n_unw before IDX. Long-format output uses the override in DECIMALS and display strings while retaining full precision in OBS_VALUE.

See Also

[tabulate_h\(\)](#), [tabulate_v\(\)](#), [write_mics_table\(\)](#)

Description

The plain UTF-8 CSV file ‘inst/extdata/survey_choices.csv’ in the package source controls the application’s survey and wave selectors. Edit this source file directly before committing to GitHub or building the package.

Columns

Each row represents one available country, period, and wave combination. All four columns are read as character strings:

country Full country name, for example Jamaica.

country_code Exactly three uppercase letters, for example JAM.

period Survey period as text, for example 2023–24.

wave Wave label, for example Wave 1.

Keep the column names. The file must have at least one row, with no empty fields or duplicate country/period/wave combinations. All waves of the same country and period must use the same country code. Leading and trailing whitespace is removed. Row order determines selector order and defaults.

Editing and installation

Use a text editor or a spreadsheet editor that preserves the period and wave columns as text. Save as UTF-8 CSV. From the package root, run `source("run_dev.R")` to load and launch the development source. Stop and restart the app after saving changes.

The CSV requires no conversion to an R data object.

R installs ‘inst/extdata’ files into the package’s ‘extdata’ directory. For installed packages, reinstall the updated source and restart the app. Direct edits to an installed copy are replaced by re-installation or upgrades. The selected code, full period, and wave also contribute to output filenames and long-format metadata.

See Also

[run_app, micsPlusTableR-package](#)

Examples

```
path <- system.file("extdata", "survey_choices.csv", package = "micsPlusTableR")
choices <- read.csv(path, colClasses = "character", fileEncoding = "UTF-8-BOM")
choices
```

tabulate_extra_table	<i>Map a supplied extra table to the current plan</i>
----------------------	---

Description

Map a supplied extra table to the current plan

Usage

```
tabulate_extra_table(session, table)
```

Arguments

session	A prepared session with a loaded plan.
table	A data frame whose first column contains row labels and whose remaining columns contain values in plan order.

Details

Row logic is retained as metadata, not evaluated against survey data. Values are mapped by position and the engine applies suppression. The plan's optional `d=` argument controls display precision for supplied values too; see [statistic-precision](#).

Value

Long-format results, also saved in `session$cell_results`.

tabulate_h	<i>Calculate a horizontal table</i>
------------	-------------------------------------

Description

Process column filter blocks and calculate column indicators using the currently loaded horizontal plan. The direction must match the plan. Results are also saved in `session$cell_results`. Use [tabulate_mics_table\(\)](#) for automatic direction selection and additional variable-explanation labels.

Usage

```
tabulate_h(session, skip_row_conditions = FALSE)
```

Arguments

session	A session created by mics_session() with prepared data and a plan loaded by read_mics_tabulation() .
skip_row_conditions	Logical; replace row predicates with TRUE during calculation while retaining their labels. Intended for extra-table testing.

Details

The condition-row `filter(...)` in Excel column B applies globally. Each additional horizontal `filter(...)` starts at its own column and combines with B. For each row independently, it extends right until the next explicit filter or the first transition from a non-mean statistic to a mean statistic. That mean cell uses B alone unless it starts a new explicit filter. A stopped filter stays off until another explicit filter starts. Mean statistics are `mean(variable)`, `mean`, and the legacy `Mean` form. Changes among other statistics (including `p`, `n`, and `n_unw`), between mean variables, or from a mean to a non-mean do not stop inheritance. Worksheet statistic filling and trimming surrounding whitespace happen before scope is determined. Changes down rows do not stop a filter. Sources, calculation chains, weights, and column/row predicates retain their existing roles; filters run before calculations. Vertical tabulation keeps its separate primary-filter behavior; see [tabulate_v\(\)](#). Display precision `d=` is independent of filter scope; see [statistic-precision](#).

Value

A long-format cell-results data frame.

See Also

[tabulate_v\(\)](#), [tabulate_mics_table\(\)](#)

<code>tabulate_mics_table</code>	<i>Tabulate a MICS Plus table outside the application</i>
----------------------------------	---

Description

Tabulate a MICS Plus table outside the application

Usage

```
tabulate_mics_table(session, path_tab_excel = NULL, sheet = NULL)
```

Arguments

- | | |
|-----------------------------|--|
| <code>session</code> | A prepared tabulation session. |
| <code>path_tab_excel</code> | Optional tabulation-plan path. Supply with <code>sheet</code> to read a new table before tabulating. |
| <code>sheet</code> | Optional sheet name or index. |

Value

A long-format data frame containing one row per calculated cell.

tabulate_v	<i>Calculate a vertical table</i>
------------	-----------------------------------

Description

Filter prepared data by column and calculate row indicators using the currently loaded vertical plan. The direction must match the plan. Results are also saved in `session$cell_results`. Use [tabulate_mics_table\(\)](#) for automatic direction selection and additional variable-explanation labels.

Usage

```
tabulate_v(session, skip_row_conditions = FALSE)
```

Arguments

session	A session created by mics_session() with prepared data and a plan loaded by read_mics_tabulation() .
skip_row_conditions	Logical; replace row predicates with TRUE during calculation while retaining their labels. Intended for extra-table testing.

Details

Every supported statistic accepts an optional `d=` display argument; for example, `mean(x, d=0)`, `mean(d=1)`, or `p(d=2)`. Omitting it preserves existing formatting. See [statistic-precision](#) for rules and examples.

Value

A long-format cell-results data frame.

See Also

[tabulate_h\(\)](#), [tabulate_mics_table\(\)](#)

validate_sdmx_compatible	<i>Validate SDMX-compatible MICS long-format data</i>
--------------------------	---

Description

Checks the complete 35-variable contract and the unique observation key: `SURVEY_ID`, `TABLE_ID`, `row_order`, `column_order`, and `TIME_PERIOD`.

Usage

```
validate_sdmx_compatible(data)
```

Arguments

data A data frame produced by `as_sdmx_compatible()`.

Value

data, invisibly.

write_mics_footnotes *Add standard footnotes to a tabulation workbook*

Description

Add standard footnotes to a tabulation workbook

Usage

```
write_mics_footnotes(session, destination, sheet = NULL, table = NULL)
```

Arguments

session A tabulation session.

destination Existing destination workbook.

sheet Sheet name or index; defaults to the current sheet.

table Cell results; defaults to the latest result.

Value

The destination path, invisibly.

write_mics_table *Write cell results to a tabulation workbook*

Description

Write cell results to a tabulation workbook

Usage

```
write_mics_table(
  session,
  destination,
  sheet = NULL,
  table = NULL,
  formatted = FALSE,
  drop_n_unw = FALSE
)
```

Arguments

session	A tabulation session with a loaded plan.
destination	Existing destination workbook.
sheet	Sheet name or index; defaults to the current sheet.
table	Cell results; defaults to the latest result.
formatted	Whether to write formatted display values.
drop_n_unw	Whether to omit unweighted-count columns when formatting.

Details

Optional worksheet d= arguments set Excel decimal places in both output formats. Numeric values retain their precision; suppression markers and parentheses retain their meaning. See [statistic-precision](#).

Value

The destination path, invisibly.

Index

- * **documentation**
 - survey_choices, 27
- * **package**
 - micsPlusTableR-package, 2
- as_sdmx_compatible, 5
- as_sdmx_compatible(), 4, 31
- calc_cells, 5
- calc_cells(), 3
- check_mics_table, 6
- check_mics_table(), 3, 8, 9, 23–25
- check_prep_dependencies, 7
- check_prep_dependencies(), 4, 14, 19
- col_condition_f, 7
- col_condition_f(), 3, 22
- col_group_perc_total_check, 8
- col_group_perc_total_check(), 4
- col_group_total_check, 9
- col_group_total_check(), 4
- compare_mics_tables, 9
- compare_mics_tables(), 3
- create_excel_workbooks, 10
- create_excel_workbooks(), 3
- download_survey_files, 11
- download_survey_files(), 15
- extract_var, 12
- extract_var(), 3
- filter_second, 13
- filter_second(), 3
- get_blank_cols, 13
- get_blank_cols(), 3
- install_prep_dependencies, 14
- install_prep_dependencies(), 4, 7, 19, 21
- list_survey_files, 15
- list_survey_files(), 11, 12
- mics_session, 16
- mics_session(), 2, 3, 21, 28, 30
- mics_table_name, 16
- micsPlusTableR
 - (micsPlusTableR-package), 2
- micsPlusTableR-package, 2, 18
- normalize_condition_text, 17
- normalize_condition_text(), 3, 8, 22
- offline-workflow, 3, 4, 17
- package-dependencies, 4, 18, 20
- pivot_mics_table, 19
- pivot_mics_table(), 3
- prep-script-dependencies, 4, 7, 14, 18, 19, 21
- prepare_mics_data, 20
- prepare_mics_data(), 3, 7, 14, 19, 20
- read_mics_tabulation, 21
- read_mics_tabulation(), 28, 30
- read_previous_mics_table, 22
- read_previous_mics_table(), 3
- row_condition_f, 22
- row_condition_f(), 3, 8
- row_group_perc_total_check, 23
- row_group_perc_total_check(), 4
- row_group_total_check, 23
- row_group_total_check(), 4
- row_indent_group_perc_total_check, 24
- row_indent_group_perc_total_check(), 4
- row_indent_group_total_check, 25
- row_indent_group_total_check(), 4
- run_app, 25, 27
- run_app(), 2–4
- statistic-precision, 6, 26, 28–30, 32
- survey_choices, 4, 27
- tabulate_extra_table, 28
- tabulate_extra_table(), 3
- tabulate_h, 28
- tabulate_h(), 3, 26, 30
- tabulate_mics_table, 29
- tabulate_mics_table(), 3, 28–30
- tabulate_v, 30

`tabulate_v()`, [3](#), [26](#), [29](#)

`validate_sdmx_compatible`, [30](#)

`validate_sdmx_compatible()`, [4](#)

`write_mics_footnotes`, [31](#)

`write_mics_footnotes()`, [3](#)

`write_mics_table`, [31](#)

`write_mics_table()`, [3](#), [26](#)